# Relatório de Auditoria de Segurança

## SISPAT-AGU

Data do relatório: 13/10/2026

Auditor: Agente de Segurança da Informação / SecOps (auditoria de código real, sem especulação)

Diretório raiz auditado: `C:\Users\winer\Videos\School\INVENTARIOAGU\SISPAT-AGU`

### Escopo auditado

• Backend Flask (Python 3.12, factory create_app, Gunicorn) com SQLAlchemy 2.x e migrações Alembic (mais espelho .sql manual).

• Banco PostgreSQL via Supabase remoto (projeto tocbtonqnahmtegfkrmn) — inspecionado via código e via estado real do banco (MCP: list_tables, get_advisors, execute_sql).

• Autenticação via Supabase Auth (JWT HS256/PyJWT) e autorização via RBAC próprio (roles, permissions, escopo por unidade).

• Frontend Next.js 14 / React 18 / TypeScript / Tailwind (sem autenticação de frontend implementada ainda).

• Infraestrutura: Docker Compose (base + override dev), ausência de CI, ausência de repositório git inicializado.

• Categorias avaliadas: (1) isolamento de dados/RLS, (2) permissão definida no cliente, (3) IDOR, (4) segredos expostos, (5) XSS.

### Nota metodológica

Todo achado foi confirmado por leitura direta do código/config/migrações no momento da auditoria e/ou por consulta ao estado real do banco Supabase via MCP. Nenhum achado é especulativo; quando uma categoria não se aplica porque a funcionalidade correspondente ainda não existe (ex.: IDOR, XSS), isso é declarado explicitamente em vez de forçar um achado artificial. O mapeamento de cada categoria para a stack real do projeto foi feito da seguinte forma:

• **Isolamento de dados (RLS/tenant):** leitura completa das migrações Alembic/SQL e consulta direta ao Supabase real via MCP (list_tables, get_advisors, execute_sql em pg_roles), cruzada com backend/app/database/session.py e backend/app/config/settings.py para confirmar qual credencial o backend usa (SQLAlchemy direto vs. API Supabase).

• **Permissão no cliente:** leitura integral do frontend Next.js (componentes de navegação/layout) cruzada com os blueprints Flask reais, para confirmar que nenhuma rota sensível depende apenas da UI.

• **IDOR:** enumeração completa (grep) de todas as rotas/blueprints Flask existentes em backend/app/ — sem amostragem.

• **Segredos expostos:** leitura integral de arquivos de configuração, Docker Compose, Dockerfiles, scripts e documentação, com grep por padrões de senha/token/chave em toda a árvore.

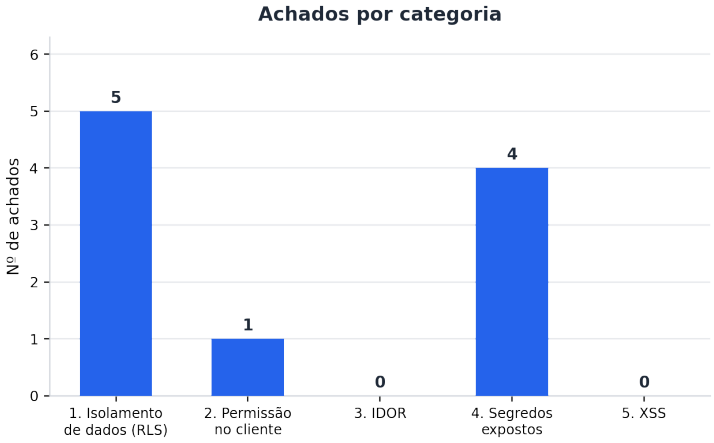
• **XSS:** grep exaustivo no frontend por dangerouslySetInnerHTML/innerHTML/eval/new Function, e no backend por render_template/Markup/Jinja.

# Resumo executivo

Esta auditoria identificou **10 achados reais** (excluindo verificações sem pendência e categorias não aplicáveis nesta fase), nenhum deles de severidade Crítica ou Alta. Duas categorias (IDOR e XSS) foram avaliadas como **não aplicáveis** nesta fase do projeto, porque a funcionalidade correspondente (endpoints de dados reais, renderização de conteúdo de usuário) ainda não existe — não por ausência de verificação.

**Achados por severidade (líquido, excl. N/A)**




*Nota sobre a contagem: a tabela-resumo de achados.md (seção 5) contém duas visões — uma contagem "bruta" por linha de tabela e uma nota de reconciliação que resolve a contagem líquida real de achados por severidade. Este relatório usa a contagem líquida reconciliada declarada no próprio documento-fonte (1 Média, 3 Baixa, 6 Informativa), excluindo a verificação SEC-2 (resultado "OK", sem achado) e as 2 categorias N/A (IDOR, XSS).*

| Severidade | Quantidade |
|---|---|
| Crítica | 0 |
| Alta | 0 |
| Média | 1 |
| Baixa | 3 |
| Informativa | 6 |
| N/A (categoria não aplicável nesta fase) | 2 |
| Verificação sem achado (SEC-2) | 1 |

# Pontos fortes

Os itens abaixo foram confirmados com evidência direta de código/config durante a auditoria (fonte: achados.md, seção 3).

- ✓ `SECRET_KEY` e `DATABASE_URL` **são obrigatórios, sem default perigoso.** `Settings` usa `Field(...)` (Pydantic, sem valor padrão) para ambos — a aplicação falha ao iniciar (`ValidationError`) se qualquer um estiver ausente do ambiente/`.env`. Evidência: `backend/app/config/settings.py:56-69`.

- ✓ `SUPABASE_JWT_SECRET` **ausente resulta em 401 explícito, nunca em bypass de autenticação.** `require_auth` verifica `if not settings.supabase_jwt_secret: ... abort(401)` antes de tentar decodificar qualquer token. Evidência: `backend/app/auth/supabase_auth.py:151-158`.

- ✓ **Nenhuma chave real em** `.env.example`**.** Todos os campos sensíveis (`DATABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_ANON_KEY`, `SECRET_KEY`, `N8N_API_KEY`) estão vazios/placeholder; os únicos valores com default inline (`${VAR:-valor}`) são operacionais: portas (`BACKEND_PORT:-8000`, `FRONTEND_PORT:-3000`), timezone (`TZ:-America/Araguaina`) e IP de proxy confiável (`TRUSTED_PROXY_IPS:-127.0.0.1`) — nenhum é um segredo.

- ✓ **Service Role Key e Anon Key nunca são efetivamente usadas em nenhum caminho de código**, nem backend nem frontend. Grep exaustivo mostra `supabase_service_role_key`/`supabase_url` apenas declarados em `Settings` e citados em docstring/README — nenhum cliente HTTP do Supabase (`supabase-js`, REST, `supabase-py`) é instanciado em lugar nenhum do repositório. O backend fala com o Postgres **diretamente** via SQLAlchemy/`DATABASE_URL`. O frontend não tem `@supabase/*` como dependência (`frontend/package.json`). Isso elimina, hoje, o cenário de risco "ANON KEY usada do lado do servidor" ou "chave privilegiada exposta ao navegador".

- ✓ **RLS habilitado em todas as 20 tabelas do schema** `public`**, confirmado no banco real** (não apenas planejado) — mesmo sem políticas, isso é fail-closed correto para qualquer role sem `BYPASSRLS` (`anon`, `authenticated`, `authenticator` — confirmado via `pg_roles`). Ver seção 1.1.

- ✓ **Token JWT e API key do n8n nunca são logados**, em nenhuma circunstância, incluindo erros. Evidência: `backend/app/auth/supabase_auth.py:173` (loga apenas `exc.__class__.__name__`); `backend/app/integrations/n8n_client.py` — toda a classe documenta e implementa isso (nenhuma linha de log inclui `n8n_api_key`/`headers`).

- ✓ **CORS implementado com allow-list explícita, sem wildcard**, cabeçalho `Vary: Origin` corretamente setado para não poluir caches/CDNs. Evidência: `backend/app/__init__.py:22-57`.

- ✓ **Nenhum detalhe interno (stack trace, SQL, caminho de arquivo) é exposto ao cliente em respostas de erro**, em nenhum status code — apenas mensagens customizadas explicitamente definidas pela aplicação são repassadas. Evidência: `backend/app/errors.py` (toda a lógica de `_is_custom_description`).

- ✓ **RBAC "negar por padrão" implementado e testado.** `require_permission`/`require_role` abortam com 401 se não há usuário autenticado, 403 se falta a permissão/papel; padrão de combinar `require_role("ADM")` + `require_permission(...)` para rotas administrativas críticas está documentado e coberto por 9 testes unitários em `backend/tests/test_rbac.py`. Evidência: `backend/app/authorization/rbac.py`, `backend/tests/test_rbac.py`.

- ✓ **Containers não executam como root** (backend: `appuser` uid 1000; frontend: `nextjs` uid 1001), builds multi-stage, portas publicadas **somente** em `127.0.0.1` (nunca `0.0.0.0`), rede `backend-net` marcada `internal: true` (Redis sem acesso externo). Evidência: `backend/Dockerfile:50-71`, `frontend/Dockerfile:47-57`, `docker-compose.yml:26-34,74-82,118-134,162-169`.

- ✓ `patrimony_number` **tratado como texto, nunca numérico.** Confirmado tanto no modelo (`backend/app/models/assets.py`, via `README.md:103`) quanto no cliente de leitura do Google Sheets, que usa `valueRenderOption="FORMATTED_VALUE"` e nunca chama `int()`/`float()` sobre a coluna `PATRIMONIO`. Evidência: `backend/app/integrations/google_sheets_client.py:43-47,319-345`.

- ✓ **Cliente Google Sheets é somente leitura** (escopo OAuth `spreadsheets.readonly`, sem nenhum método de escrita) e nunca loga o conteúdo do arquivo de credenciais da service account. Evidência: `backend/app/integrations/google_sheets_client.py:1-45,208-233`.

- ✓ **Nenhum** `dangerouslySetInnerHTML`, `innerHTML`, `eval(` **ou** `new Function(` **em todo** `frontend/src/` (grep exaustivo, zero ocorrências) — toda renderização passa por interpolação JSX, que o React escapa automaticamente por padrão.
- ✓ **Intenção correta documentada para token de sessão no frontend**: o comentário em `api-client.ts` já instrui explicitamente a nunca guardar credenciais em `localStorage`, preferindo cookie `httpOnly` — mesmo a feature ainda não estando implementada. Evidência: `frontend/src/lib/api-client.ts:8-13,65-74`.
- ✓ `.gitignore` **cobre corretamente os padrões de segredo** (`.env`, `.env.*` exceto `.env.example`, `*.pem`, `*.key`, `google_sheets_credentials*.json`) — mesmo não havendo repositório git real hoje, a intenção está correta para quando um `git init` ocorrer.
- ✓ **Testes usam apenas credenciais fictícias e obviamente falsas** (`test-secret-key`, `test-jwt-secret`, `postgresql+psycopg2://user:pass@localhost:5432/test`), nunca valores reais. Evidência: `backend/tests/conftest.py:17-19`.
- ✓ **Princípio "frontend nunca é mecanismo de segurança" e "nunca confiar em perfil/permissão enviados pelo cliente" documentados explicitamente como regra de projeto** em `CLAUDE.md:18,26,42`, antecipando corretamente a categoria 2 desta auditoria.

# Pontos fracos (riscos centrais reais)

Diferente dos pontos fortes, os riscos centrais reais identificados nesta auditoria são **poucos e de severidade limitada** — não há achados Crítica ou Alta. Os itens abaixo resumem apenas os riscos de maior relevância relativa (Média e Baixa); os demais achados (Informativa) são melhorias de robustez/documentação, não vulnerabilidades exploráveis hoje, e estão detalhados na seção de achados.

**Média** RLS-1 — RLS habilitado em produção (Supabase), mas não reproduzível via migração versionada. Se o schema for recriado a partir das migrações do repositório, o RLS não será habilitado automaticamente, revertendo silenciosamente a postura fail-closed.

**Baixa** SEC-1 — Default de senha previsível (sispat_dev) para o Postgres de desenvolvimento local, mitigado por profile explícito e bind em 127.0.0.1, mas ainda um padrão reutilizável por engano.

**Baixa** SEC-5 — ProxyFix confia incondicionalmente nos cabeçalhos X-Forwarded-* sem validar o IP de origem, mitigado hoje pela topologia de rede (bind em 127.0.0.1), mas frágil a mudanças futuras de topologia.

**Baixa** RLS-4 — Extensão pg_trgm instalada no schema public em vez de um schema dedicado (boa prática do Supabase, sem exploração identificada).

# Achados detalhados por categoria

## Categoria 1 — Isolamento de dados (RLS/tenant)

| Severidade | Arquivo:linha | Descrição |
|---|---|---|
| Média | `backend/migrations/versions/0001_initial_schema.py`, `backend/migrations/versions/0002_import_previews.py`, `backend/migrations/sql/0001_initial_schema.sql`, `backend/migrations/sql/0002_seeds_roles_permissions.sql`, `backend/migrations/sql/0003_import_previews.sql` — nenhum contém `ENABLE ROW LEVEL SECURITY`, `FORCE ROW LEVEL SECURITY` ou `CREATE POLICY` (busca exaustiva, zero ocorrências). | RLS-1 — RLS habilitado em produção (Supabase), mas não reproduzível via migração versionada (drift de infraestrutura) — o RLS que protege o banco hoje foi aplicado **fora do pipeline de migração** (manualmente, via dashboard/SQL ad-hoc no Supabase, ou por outro agente/processo não documentado neste repositório). Se o schema for recriado a partir das migrações deste repositório (cenário plausível: novo ambiente, disaster recovery, ou o próprio fluxo documentado em `backend/README.md` — "Aplicando o schema manualmente via Supabase... quando não houver conectividade Python → Postgres"), o RLS **não será habilitado**, revertendo silenciosamente a postura fail-closed para fail-open em qualquer role sem `BYPASSRLS`. |
| Informativa | `mcp__supabase__get_advisors(security)` — 20 achados `rls_enabled_no_policy`, nível **`INFO`** (não WARN/ERROR) na própria classificação do linter do Supabase. | RLS-2 — Zero políticas RLS criadas (apenas `ENABLE`, sem `CREATE POLICY`) — com RLS habilitado e zero políticas, o comportamento padrão do Postgres é **negar todo acesso** a quem não tem `BYPASSRLS` — ou seja, esse é exatamente o "fail-closed" que a tarefa pediu para confirmar. É seguro **apenas enquanto** (a) o backend continuar conectando com uma role `BYPASSRLS` (confirmado: `postgres`/`service_role`) e (b) nenhuma role `anon`/`authenticated` passar a ser usada para acesso direto (confirmado: nenhum uso de `supabase-js`/API REST hoje). Ambas as condições estão confirmadas como verdadeiras no código atual. |
| Informativa | `backend/app/authorization/rbac.py:116-134` — função `has_unit_access(user, unit_id)` consulta `UserUnitAccess` e está documentada como "preparada para uso futuro por endpoints com escopo de unidade... o módulo de importação desta fase não depende de escopo de unidade." | RLS-3 — Escopo por unidade (`user_unit_access`) implementado como utilitário, mas não aplicado em nenhuma rota (porque nenhuma rota de dados existe ainda) — não existe hoje nenhum endpoint Flask que leia/escreva `assets`, `inventories`, `units`, etc. (ver Categoria 3 — IDOR, N/A). Portanto não há "exploração" possível — mas é um lembrete explícito para a próxima fase: **qualquer rota que listar/alterar bens, inventários ou pessoas responsáveis deve chamar `has_unit_access` (ou equivalente) antes de retornar/alterar dados**, já que a tabela e a função utilitária já existem mas nenhuma rota as usa ainda. |
| Baixa | `mcp__supabase__get_advisors(security)` — achado `extension_in_public`, nível WARN, 1 ocorrência (`pg_trgm`). | RLS-4 — Extensão `pg_trgm` instalada no schema `public` (boas práticas) — extensões no schema `public` podem, em cenários específicos, ser alvo de ataques de "search_path hijacking" se funções SECURITY DEFINER existirem sem `search_path` fixo. Não há evidência de que isso seja explorável hoje (nenhuma função SECURITY DEFINER foi encontrada no schema), mas é a boa prática recomendada pelo próprio Supabase. |

| Severidade | Arquivo:linha | Descrição |
|---|---|---|
| Informativa | `backend/app/auth/supabase_auth.py:22-37` | Nota informativa — Limitação documentada de HS256 vs. JWKS/RS256 — A implementação atual assume que o projeto Supabase usa o segredo simétrico legado (HS256). Se o projeto migrar para JWKS/RS256 (comum em projetos Supabase mais novos), a verificação de assinatura atual deixaria de funcionar (falha seguro: tokens seriam rejeitados com 401, não aceitos incorretamente) — já documentado como pendência conhecida no próprio código, não implementado. Não é uma falha de segurança (o comportamento em caso de incompatibilidade é negar acesso, não conceder), apenas uma lacuna funcional a resolver antes de produção. |

## Categoria 2 — Permissão definida no cliente

| Severidade | Arquivo:linha | Descrição |
|---|---|---|
| Informativa | – | PERM-1 — Navegação do frontend exibe todos os itens (incluindo administrativos) incondicionalmente; nenhuma lógica de RBAC no cliente ainda — não há dado sensível exposto hoje — mostrar um item de menu que leva a uma página "Em construção" não é, por si, uma violação de confidencialidade. O risco é **prospectivo**: se uma página administrativa real for implementada antes de (a) existir autenticação de frontend e (b) o endpoint Flask correspondente aplicar `require_auth`/`require_permission`/`require_role`, a única barreira seria a UI — o que o próprio `CLAUDE.md` já proíbe explicitamente ("nunca confiar em perfil/permissão enviados pelo cliente", linha 26). |

## Categoria 3 — IDOR

Categoria não aplicável nesta fase (ver detalhes na seção de rascunho/achados-fonte) — nenhuma vulnerabilidade a listar porque a funcionalidade correspondente ainda não existe.

## Categoria 4 — Segredos expostos

| Severidade | Arquivo:linha | Descrição |
|---|---|---|
| Baixa | `docker-compose.dev.yml:93-95` | SEC-1 — Defaults de credencial (não-secreta, mas previsível) para o Postgres de desenvolvimento local — diferente dos defaults de porta/timezone (`BACKEND_PORT:-8000`, `TZ:-America/Araguaina`), este **é** um default de credencial (usuário/senha), o que a tarefa pediu para tratar com atenção redobrada. Avaliando o contexto real, porém: |

| Severidade | Arquivo:linha | Descrição |
|---|---|---|
| Informativa | `git status`, `git log --oneline`, `git remote -v` executados na raiz do projeto retornam `fatal: not a git repository (or any of the parent directories): .git`. | SEC-3 — Ausência de repositório git (impossibilita auditoria de histórico, mas também significa que nada foi commitado/vazado até agora) — a tarefa pediu para confirmar explicitamente se há um repositório git real antes de afirmar "nenhum segredo commitado". A resposta correta e honesta é: **não há commits a auditar porque não há repositório git inicializado**. Isso não é, em si, uma vulnerabilidade — é, na verdade, a situação mais segura possível (zero superfície de histórico exposta) — mas também significa que a garantia "nenhum segredo foi commitado" não pode ser verificada por grep de histórico; ela repousa inteiramente na disciplina atual de `.gitignore` e no fato de `.env` nunca ter sido criado neste ambiente. |
| Informativa | `backend/app/config/settings.py` não declara nenhum campo `google_sheets_credentials_json`/`google_sheets_spreadsheet_id` (confirmado por leitura integral do arquivo), enquanto `backend/app/integrations/google_sheets_client.py:185-201` acessa esses atributos via `getattr(self._settings, "...", None)` — deliberadamente tolerante "para não acoplar este cliente à declaração exata desses campos em `Settings`" (docstring do próprio módulo, linhas 25-30). | SEC-4 — `GOOGLE_SHEETS_CREDENTIALS_JSON`/`GOOGLE_SHEETS_SPREADSHEET_ID` não declarados em `Settings`, apenas lidos via `getattr` tolerante — o efeito prático é fail-closed — sem os campos declarados em `Settings`, `getattr` sempre retorna `None`, e `GoogleSheetsClient` sempre lança `GoogleSheetsClientError` em vez de tentar operar com um caminho de credencial ausente/incorreto. Não há vazamento, apenas uma integração que ainda não funciona de ponta a ponta — um gap de integração entre duas frentes de trabalho em paralelo, não uma vulnerabilidade. |
| Baixa | `backend/app/__init__.py:80-104` — o próprio código documenta extensivamente que `ProxyFix` confia incondicionalmente no cabeçalho `X-Forwarded-For`/`X-Forwarded-Proto`/`X-Forwarded-Host` de quem quer que fale diretamente com o processo Flask, sem checar `TRUSTED_PROXY_IPS` contra o IP real do peer. | SEC-5 — Confiança no `ProxyFix` sem validação de IP de origem (já autodocumentada) — a porta do backend só é publicada em `127.0.0.1` (`docker-compose.yml:81-82`), de modo que, na topologia documentada (CloudPanel como único proxy na mesma VPS), não há caminho de rede para um atacante falar diretamente com o Flask sem passar pelo CloudPanel. |

## Categoria 5 — XSS

Categoria não aplicável nesta fase (ver detalhes na seção de rascunho/achados-fonte) — nenhuma vulnerabilidade a listar porque a funcionalidade correspondente ainda não existe.

# Recomendações priorizadas

*Extraídas de achados.md, seção 6 — organizadas por prioridade de implementação.*

## P1 — fazer antes de implementar os primeiros endpoints de dados reais — `assets`/`units`/`inventories`/`imports`

• Ao criar o primeiro endpoint que recebe um ID (path/query/body) para ler/alterar/deletar um registro, aplicar desde o primeiro commit: `@require_auth` + verificação de posse/escopo via `has_unit_access` (ou equivalente) — reauditar a categoria IDOR nesse momento (ver IDOR, RLS-3).

• Antes de qualquer página de `/administracao/*` passar a buscar dados reais no frontend, confirmar que o endpoint Flask correspondente já aplica `require_auth`/`require_permission`/`require_role` — nunca depender apenas de esconder o item de menu (ver PERM-1).

• Versionar o `ENABLE ROW LEVEL SECURITY` das 20 tabelas em uma migração Alembic (+ espelho `.sql`), para que o estado de segurança do Supabase real seja reproduzível a partir do repositório (ver RLS-1).

## P2 — antes de produção / próxima rodada de hardening

• Remover o default de senha (`sispat_dev`) do `docker-compose.dev.yml`/`.env.example` para o Postgres de desenvolvimento local, exigindo valor explícito sem fallback (ver SEC-1).

• Mover a extensão `pg_trgm` para um schema dedicado fora de `public` (ver RLS-4).

• Configurar CI básico (lint + testes + scanning de segredos) assim que o repositório git for inicializado — hoje não há `.github/workflows/` nem qualquer outro CI (ver seção 1, SEC-3).

• Declarar `google_sheets_credentials_json`/`google_sheets_spreadsheet_id` explicitamente em `Settings` em vez de `getattr` tolerante, quando a integração for finalizada (ver SEC-4).

## P3 — acompanhar, não bloqueante

• Reavaliar a verificação JWT (HS256 vs. JWKS/RS256) antes de produção, confirmando qual esquema o projeto Supabase real usa (ver nota HS256/JWKS).

• Ao adicionar qualquer renderização de conteúdo de usuário em HTML (frontend ou e-mail/relatório no backend), usar sanitização explícita (ex. DOMPurify) e nunca `dangerouslySetInnerHTML` direto — reauditar XSS nesse momento.

• Considerar validação de IP de origem para os cabeçalhos de proxy confiados pelo `ProxyFix`, caso a topologia de rede documentada mude (ver SEC-5).

# Issues para o GitHub

*Rascunhos de issues extraídos integralmente de achados.md, seção 7, preservando os delimitadores `--- ISSUE n ---` / `--- FIM ISSUE n ---`.*

**--- ISSUE 1 ---**

## [Segurança] Versionar `ENABLE ROW LEVEL SECURITY` das 20 tabelas em migração Alembic

**Labels sugeridas:**

`security`, `severity:medium`, `infra`, `database`

**Descrição:**

O projeto Supabase real (`tocbtonqnahmtegfkrmn`) tem RLS habilitado em todas as 20 tabelas do schema `public`, confirmado via `mcp__supabase__list_tables` (`rowsecurity: true` em todas) e `mcp__supabase__get_advisors(security)` (20 achados `rls_enabled_no_policy`, nível INFO — esperado nesta fase, já que o backend acessa via role com `BYPASSRLS`). Porém, nenhuma das migrações versionadas no repositório contém a instrução que habilitou esse RLS.

**Evidência:**

- Busca exaustiva por `ENABLE ROW LEVEL SECURITY`/`FORCE ROW LEVEL SECURITY`/`CREATE POLICY` em `backend/migrations/versions/*.py` e `backend/migrations/sql/*.sql`: zero ocorrências.
- Estado real do banco (via MCP Supabase), tabela de exemplo: `public.assets` → `rls_enabled: true`, mas sem política.

**Impacto:**

Se o schema for recriado a partir das migrações deste repositório (novo ambiente, disaster recovery), o RLS não será habilitado automaticamente, revertendo silenciosamente para fail-open em qualquer role sem `BYPASSRLS` (ex.: `anon`, `authenticated`). Não é explorável hoje (o banco real já tem RLS habilitado e o backend usa role com bypass), mas é uma lacuna de infraestrutura-como-código.

**Sugestão de correção:**

Criar uma migração Alembic (`alembic revision`) que execute `ALTER TABLE <tabela> ENABLE ROW LEVEL SECURITY;` para as 20 tabelas do schema `public`, replicada no espelho `backend/migrations/sql/`.

**Critérios de aceite:**

- ■ Nova migração Alembic criada, habilitando RLS nas 20 tabelas.
- ■ Espelho `.sql` correspondente atualizado (`backend/migrations/sql/`).
- ■ `alembic upgrade head` em um banco limpo resulta no mesmo estado de RLS hoje observado no Supabase real (verificável via `mcp__supabase__list_tables` ou `SELECT tablename, rowsecurity FROM pg_tables WHERE schemaname='public'`).
- ■ `backend/README.md` atualizado para não mais afirmar "o controle de acesso por unidade é aplicado pelo backend, não por RLS" sem mencionar que RLS está habilitado como camada adicional fail-closed.

**--- FIM ISSUE 1 ---**

**--- ISSUE 2 ---**

## [Segurança] Aplicar escopo de unidade (`user_unit_access`) em todo endpoint futuro que leia/altere bens, inventários ou pessoas responsáveis

**Labels sugeridas:**

`security`, `severity:info`, `tracked-risk`, `backend`

**Descrição:**

A função utilitária `has_unit_access(user, unit_id)` já existe e está testada (`backend/tests/test_rbac.py`), mas não há, até o momento desta auditoria, nenhum endpoint Flask em `backend/app/api/` que a utilize — porque nenhum endpoint de dados (`assets`, `units`, `inventories`, `imports`) foi implementado ainda. Esta issue existe para rastrear a

exigência antes que esses endpoints sejam escritos.

**Evidência:**

- `backend/app/authorization/rbac.py:116-134` — `has_unit_access` definida, comentário explícito: "função utilitária preparada para uso futuro... o módulo de importação desta fase não depende de escopo de unidade."
- Grep por `has_unit_access(` em `backend/` — nenhuma chamada fora da própria definição e de `backend/tests/test_rbac.py`.

**Impacto:**

Risco rastreado, não uma vulnerabilidade em código existente. Se os primeiros endpoints de dados forem implementados sem aplicar escopo de unidade, um usuário com papel `USER`/`RH` de uma unidade poderia ler/alterar dados de outra unidade (IDOR por ausência de verificação de escopo, não por ausência de verificação de ID).

**Sugestão de correção:**

Ao implementar o primeiro endpoint que lista/altera `assets`/`units`/`inventories`/`responsible_people`, aplicar `has_unit_access` (ou filtro de query equivalente) antes de retornar/persistir dados, para papéis que não sejam `ADM`.

**Critérios de aceite:**

- ■ Todo novo endpoint de leitura/escrita de dados escopados por unidade chama `has_unit_access` (ou filtra a query por `user_unit_access`) antes de agir.
- ■ Teste de regressão: usuário sem vínculo em `user_unit_access` para a unidade do recurso recebe 403 (ou lista vazia, conforme o padrão de resposta escolhido).
- ■ Este padrão documentado em `backend/README.md` ou docstring do módulo de rotas correspondente.

**--- FIM ISSUE 2 ---**

**--- ISSUE 3 ---**

## [Segurança] Garantir enforcement no backend antes de qualquer página de `/administracao/*` exibir dados reais

**Labels sugeridas:**

`security`, `severity:info`, `tracked-risk`, `frontend`, `backend`

**Descrição:**

`frontend/src/components/layout/nav-items.ts` e `frontend/src/components/layout/sidebar.tsx` documentam explicitamente (comentário `TODO`) que todos os itens de navegação, incluindo o grupo "Administração" (Usuários, Perfis e Permissões, Vínculos de Unidades, Auditoria, Integrações, Importação Google Sheets, Configurações do Sistema), são exibidos incondicionalmente, sem RBAC no cliente. Hoje isso não expõe dado nenhum (as páginas correspondentes não existem ou são placeholders estáticos), mas é um ponto a travar antes de dados reais existirem.

**Evidência:**

- `frontend/src/components/layout/nav-items.ts:30-34` (comentário TODO).
- `frontend/src/components/layout/sidebar.tsx:79-81` (comentário TODO idêntico).
- `frontend/src/app/administracao/importacao/google-sheets/page.tsx:11-19` — única página administrativa implementada, é um placeholder estático "Em construção", sem chamada a `apiClient`.
- Cruzamento com backend: nenhum endpoint Flask existe ainda para nenhuma dessas rotas administrativas (confirmado — ver Issue/achado de IDOR, N/A).

**Impacto:**

Nenhum hoje (nenhum dado real é exibido). Risco prospectivo: se uma dessas páginas passar a buscar dados reais antes do endpoint Flask correspondente aplicar `require_auth`/`require_permission`/`require_role`, a única barreira seria a visibilidade da UI — insuficiente como controle de segurança (já proibido explicitamente por `CLAUDE.md`, linha 26: "Nunca confiar em perfil/permissão enviados pelo cliente").

**Sugestão de correção:**

Ao implementar cada página administrativa real, checklist obrigatório: (1) o endpoint Flask correspondente já usa `@require_auth` + `@require_permission`/`@require_role`; (2) a visibilidade do item de menu no frontend é apenas

cosmética, nunca a única proteção.

**Critérios de aceite:**

- ■ Antes de cada página de `/administracao/*` ser implementada com dados reais, o endpoint Flask correspondente tem decorator de autenticação/autorização aplicado e testado.
- ■ TODO em `nav-items.ts`/`sidebar.tsx` substituído por lógica real de visibilidade condicionada a permissões, quando a autenticação de frontend existir — sem remover a exigência de enforcement no backend.

**--- FIM ISSUE 3 ---**

**--- ISSUE 4 ---**

## [Segurança] Remover default de senha previsível (`sispat_dev`) do Postgres de desenvolvimento local

**Labels sugeridas:**

`security`, `severity:low`, `infra`, `good-practice`

**Descrição:**

`docker-compose.dev.yml` (serviço `postgres-dev`, profile `dev-db`) e `.env.example` definem `POSTGRES_DEV_USER`/`POSTGRES_DEV_PASSWORD`/`POSTGRES_DEV_DB` com o mesmo valor de fallback (`sispat_dev`) via sintaxe `${VAR:-sispat_dev}`. Diferente dos demais defaults do projeto (portas, timezone), este é um default de credencial.

**Evidência:**

```
# docker-compose.dev.yml:93-95
environment:
  - POSTGRES_USER=${POSTGRES_DEV_USER:-sispat_dev}
  - POSTGRES_PASSWORD=${POSTGRES_DEV_PASSWORD:-sispat_dev}
  - POSTGRES_DB=${POSTGRES_DEV_DB:-sispat_dev}
```

**Impacto:**

Baixo no contexto atual: o serviço só sobe com `--profile dev-db` explícito, é documentado como "NÃO é a fonte de verdade", e a porta é publicada apenas em `127.0.0.1`. O risco é de reaproveitamento acidental do padrão (copiar para um ambiente acessível pela rede, ou reusar o mesmo valor em outro lugar).

**Sugestão de correção:**

Remover o fallback de `POSTGRES_DEV_PASSWORD` (exigir que a variável seja definida explicitamente, sem default) ou gerar um valor aleatório em `scripts/init.sh` na primeira execução.

**Critérios de aceite:**

- ■ `POSTGRES_DEV_PASSWORD` não tem mais fallback hardcoded em `docker-compose.dev.yml`.
- ■ `.env.example` deixa claro que o valor deve ser definido pelo desenvolvedor, ou `scripts/init.sh` passa a gerar um valor aleatório automaticamente.

**--- FIM ISSUE 4 ---**

**--- ISSUE 5 ---**

## [Segurança] Mover extensão `pg_trgm` para schema dedicado (fora de `public`)

**Labels sugeridas:**

`security`, `severity:low`, `database`, `good-practice`

**Descrição:**

O advisor de segurança do Supabase (`mcp__supabase__get_advisors(type="security")`) reporta a extensão `pg_trgm` instalada no schema `public`, classificada como WARN (`extension_in_public`).

**Evidência:**

- `backend/migrations/sql/0001_initial_schema.sql:20`: `CREATE EXTENSION IF NOT EXISTS pg_trgm;` sem schema explícito (cai em `public` por padrão).
- Advisor Supabase: `extension_in_public`, 1 ocorrência, nível WARN.

**Impacto:**

Baixo — nenhuma função `SECURITY DEFINER` vulnerável a "search_path hijacking" foi identificada usando essa extensão nesta auditoria. É uma recomendação de boas práticas do próprio Supabase, preventiva.

**Sugestão de correção:**

Criar um schema dedicado (ex.: `extensions`) e mover `pg_trgm` para ele em uma migração futura (`ALTER EXTENSION pg_trgm SET SCHEMA extensions;`), ajustando `search_path` onde necessário.

**Critérios de aceite:**

- ■ `pg_trgm` deixa de aparecer no advisor `extension_in_public`.
- ■ Buscas textuais (GIN trigram) que dependem da extensão continuam funcionando após a migração de schema.

**--- FIM ISSUE 5 ---**

**--- ISSUE 6 ---**

## [Segurança] Configurar CI mínimo (lint, testes, scanning de segredos) ao inicializar o repositório git

**Labels sugeridas:**

`security`, `severity:info`, `ci`, `infra`

**Descrição:**

O projeto hoje não é um repositório git (`git status` retorna "not a git repository") e não há `.github/workflows/` nem qualquer outro CI configurado. Isso significa que, quando o repositório for inicializado e eventualmente conectado a um remoto (GitHub), não há nenhuma rede de proteção automática contra commit acidental de segredos ou regressão de testes.

**Evidência:**

- `git status`/`git log --oneline`/`git remote -v` executados na raiz: `fatal: not a git repository (or any of the parent directories): .git`.
- Busca por `.github/workflows/` e qualquer outro `*.yml` de CI na raiz: nenhum encontrado (os únicos `.yml` do projeto são `docker-compose.yml`/`docker-compose.dev.yml`, que não são CI).

**Impacto:**

Informativo — não há exposição hoje porque não há histórico git. O risco surge no momento em que o repositório for inicializado e o primeiro push ocorrer sem rede de proteção.

**Sugestão de correção:**

Ao rodar `git init` + primeiro commit, configurar um workflow mínimo de CI (ex. GitHub Actions) com: lint (backend `ruff`/`flake8`, frontend `eslint`/`tsc --noEmit`), `pytest` do backend, e uma etapa de scanning de segredos (ex. `gitleaks`) antes de qualquer push a um remoto compartilhado.

**Critérios de aceite:**

- ■ `.github/workflows/ci.yml` (ou equivalente) criado, executando lint + testes em PRs.
- ■ Etapa de scanning de segredos (`gitleaks`/`trufflehog`) incluída no CI ou como hook de pre-commit.
- ■ Documentado em `README.md`/`CLAUDE.md` que push a um remoto compartilhado não deve ocorrer sem essa etapa configurada.

**--- FIM ISSUE 6 ---**

**--- ISSUE 7 ---**

## [Segurança] Declarar `GOOGLE_SHEETS_CREDENTIALS_JSON`/`GOOGLE_SHEETS_SPREADSHEET_ID` explicitamente em `Settings`

**Labels sugeridas:**

`security`, `severity:info`, `backend`, `tech-debt`

**Descrição:**

`backend/app/integrations/google_sheets_client.py` lê `google_sheets_credentials_json`/`google_sheets_spreadsheet_id` via `getattr(self._settings, "...", None)`, deliberadamente tolerante porque esses campos não estão declarados em `backend/app/config/settings.py` (confirmado por leitura integral do arquivo — apenas `supabase_*`, `n8n_*`, `cors_*`, `trusted_proxy_*`, `database_url`, `secret_key`, `redis_url`/`celery_*` estão declarados).

**Evidência:**

- `backend/app/integrations/google_sheets_client.py:185-201` (`_credentials_path`/`_spreadsheet_id`, ambos via `getattr`).
- `backend/app/config/settings.py` (leitura integral): nenhum campo `google_sheets_*` declarado na classe `Settings`.

**Impacto:**

Nenhum risco de segurança direto — o efeito é fail-closed (sempre levanta `GoogleSheetsClientError` hoje, já que os campos nunca são populados a partir do `.env`). É um gap de integração entre frentes de trabalho em paralelo (backend/config vs. backend/integrations), não uma vulnerabilidade.

**Sugestão de correção:**

Declarar os dois campos (`google_sheets_credentials_json: str | None`, `google_sheets_spreadsheet_id: str | None`, e opcionalmente `google_sheets_sheet_name`/`google_sheets_sheet_gid`) explicitamente em `Settings`, para que populem corretamente a partir de `.env`/variáveis de ambiente e apareçam na documentação de configuração do módulo.

**Critérios de aceite:**

- ■ Campos declarados em `Settings` com os tipos corretos.
- ■ `GoogleSheetsClient` passa a funcionar fim-a-fim com um `.env` preenchido corretamente (testável manualmente ou via teste de integração).
- ■ Docstring do módulo `settings.py` atualizada para listar as novas variáveis suportadas.

**--- FIM ISSUE 7 ---**

**--- ISSUE 8 ---**

## [Segurança] Confirmar esquema de assinatura JWT real do projeto Supabase (HS256 legado vs. JWKS/RS256) antes de produção

**Labels sugeridas:**

`security`, `severity:low`, `auth`, `tech-debt`

**Descrição:**

`backend/app/auth/supabase_auth.py` implementa verificação de JWT assumindo o esquema legado HS256 com segredo simétrico (`SUPABASE_JWT_SECRET`). O próprio módulo documenta que projetos Supabase mais novos podem usar JWKS/RS256, caso em que a verificação precisaria ser adaptada (ler `kid`, buscar chave pública via endpoint JWKS, cachear com TTL).

**Evidência:**

- `backend/app/auth/supabase_auth.py:22-37` (docstring "LIMITAÇÃO DOCUMENTADA").

**Impacto:**

Baixo/funcional, não uma vulnerabilidade de segurança em si — se o projeto Supabase real usar JWKS/RS256 e não HS256, a verificação atual simplesmente rejeitaria todos os tokens (401 para todo usuário), falha segura, não bypass. Ainda assim, deve ser confirmado e resolvido antes de produção para não bloquear login de todos os usuários.

**Sugestão de correção:**

Confirmar no painel do Supabase (Project Settings > API > JWT Settings) qual esquema o projeto `tocbtonqnahmtegfkrmn` usa. Se for JWKS/RS256, implementar a adaptação documentada (cache de chave pública por `kid`, invalidação em caso de `kid` desconhecido).

**Critérios de aceite:**

- ■ Esquema de assinatura do projeto Supabase real confirmado e documentado.
- ■ Se JWKS/RS256: implementação adaptada e testada; se HS256: documentação atualizada confirmando que a implementação atual é definitiva.

**--- FIM ISSUE 8 ---**

**--- ISSUE 9 ---**

## [Segurança] Sanitizar qualquer renderização futura de conteúdo de usuário (frontend HTML / e-mails no backend)

**Labels sugeridas:**

`security`, `severity:info`, `xss`, `frontend`, `backend`

**Descrição:**

Auditoria de XSS concluiu "não aplicável" nesta fase: nenhum `dangerouslySetInnerHTML`/`innerHTML`/`eval`/`new Function` existe em `frontend/src/`, e o backend não usa `render_template`/Jinja para gerar HTML. Esta issue existe para garantir que, quando qualquer um desses pontos for introduzido (ex.: descrição de bem renderizada com formatação rica, e-mail de notificação, relatório HTML/PDF), sanitização adequada seja aplicada desde o primeiro commit.

**Evidência:**

- • Grep exaustivo em `frontend/src/`: zero ocorrências de `dangerouslySetInnerHTML`/`innerHTML`/`eval(`/`new Function(`.
- • Grep em `backend/app/`: zero ocorrências de `render_template`/`Markup`/Jinja.
- • `frontend/package.json`: nenhuma lib de sanitização (`dompurify` ou similar) instalada — coerente com não haver ainda nenhum ponto de risco.

**Impacto:**

Nenhum hoje. Risco prospectivo quando conteúdo de usuário passar a ser renderizado como HTML.

**Sugestão de correção:**

Ao introduzir qualquer renderização de HTML a partir de entrada do usuário (frontend ou e-mail/relatório no backend), usar uma biblioteca de sanitização (ex. DOMPurify no frontend) e nunca inserir HTML não escapado sem passar por ela.

**Critérios de aceite:**

- ■ Checklist de PR atualizado para exigir revisão de segurança sempre que `dangerouslySetInnerHTML`, `innerHTML`, ou geração de HTML no backend for introduzida.
- ■ Biblioteca de sanitização adicionada e aplicada no primeiro ponto real que a exigir.

**--- FIM ISSUE 9 ---**